



Instituto Superior de Engenharia de Lisboa

Computer Vision and Mixed Reality

Master in Informatics and Multimedia Engineering

1st Project – Feature Based Face Detection and Recognition for Augmented Reality

April 2024

Sofia Condesso — 50308

Lectured by
Professor Engineer Pedro Jorge

Contents

1	Introduction	3
2	Development	3
3	Conclusion	18
	References	18

1 Introduction

This project aims to explore and implement **face detection** and **face recognition algorithms** using various techniques. For face detection is used Haar Cascade Classifier, OpenCV Deep Neural Network module, Dlib toolbox and Mediapipe library, from an image or from webcam in real-time. For **face recognition**, **Eigenfaces** and **Fisherfaces** algorithms will be implemented and, then, a **Nearest-Neighbour classifier** will be developed to recognize faces based on both algorithms. In the end, this project has a part where the student is invited to **combine real and virtual objects**, performing virtual object **normalization and superimposition**.

a. Code organization

- In the file `project1.ipynb` it's possible to verify every module running, to **answer the exercises**, with exception for the exercise 2.a).
- All the exercises from 2.a) can be executed for **real-time detection using webcam**, in the respective scripts.

b. Technical Requirements

Due to `dlib` library, we'll need to use Python 3.8 and make the following installations (tested on MacOS):

```
pip install tqdm - for loading bars
pip install numpy
pip install matplotlib
pip install scikit-learn
pip install opencv-python
conda install -c conda-forge dlib [1]
pip install mediapipe
```

2 Development

a. Face detection

Use one of these algorithms to implement face detection in images collected in real time by a camera (see [1,2] for documentation and [3,4] as examples of implementation).

To better explore the libraries, it was decided to implement all the techniques. The python scripts can be executed to run the face detection in real-time, from the webcam. The functions in each script, are used in the Jupyter notebook `project1.ipynb`, to demonstrate with a test image.

i. Haar Cascade Classifier

OpenCV “CascadeClassifier” method is based on the algorithm of object detection using Haar feature-based cascade classifiers proposed by Paul Viola and Michael Jones in their paper, “Rapid Object Detection using a Boosted Cascade of Simple Features” in 2001.

Haar Cascade Classifier is a machine learning algorithm developed by Paul Viola and Michael Jones in 2001[2], which is used to detect faces or objects in digital images or sequences of images (videos).

It uses a large number of simple features (Haar-like features) to identify specific patterns in the image data. These features are basic image filters that capture variations in pixel intensity within rectangular regions of an image. By strategically combining these features in a hierarchical cascade, the classifier can efficiently eliminate irrelevant regions and focus on areas with higher probability of containing the target object, in this case, the face.

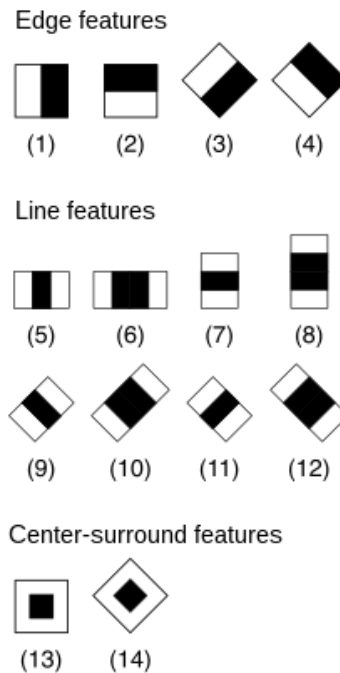


Figure 1: Haar features

To perform face detection in real time, the following steps were needed:

1. Load the pre-trained Haar Cascade Classifier model, instantiating an object from the class `cv2.CascadeClassifier()` and specifying the XML file `haarcascade_frontalface_default.xml` located at `cv2.data.haarcascades`.

2. The `detect_faces()` function takes an image as argument, converts it to grayscale using `cv2.cvtColor()` with the flag `cv2.COLOR_BGR2GRAY`, and then applies the Haar Cascade Classifier algorithm using `detectMultiScale()` function. This function detects faces in the image with a specified scale factor, minimum neighbors, and minimum size.
3. In the main loop, we continuously capture frames from the camera using `cap.read()`, apply the `detect_faces()` function to detect faces in the captured frame, display the resulting image using `cv2.imshow()`, which will be outputting the webcam video with a square around the existent faces.

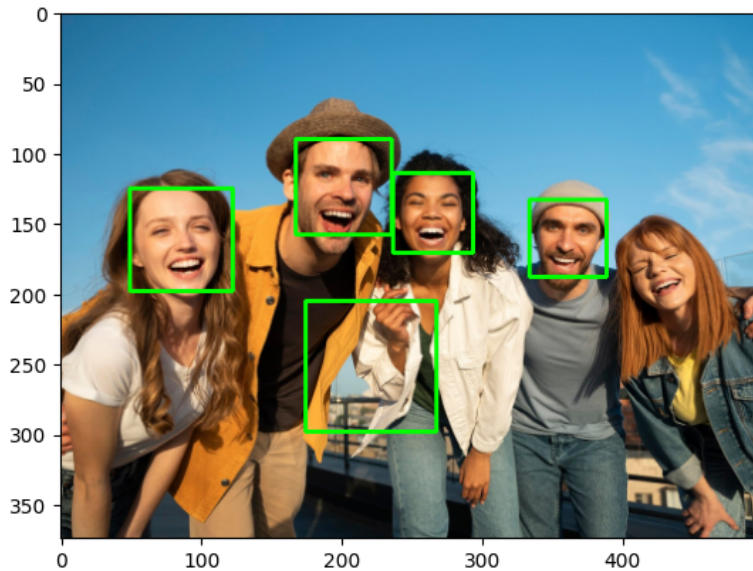


Figure 2: Face detection with Haar Cascade classifier

By observing the drawn rectangles in the test image, we verify that this approach outputs one misdetection and one False Positive.

The cascaded structure allows for rapid feature evaluation, making it suitable for real-time applications like face detection in video streams. While efficient, Haar Cascades may not achieve the highest accuracy compared to more complex deep learning models.

ii. OpenCV Deep Neural Network module (dnn)

OpenCV has a deep neural network module and includes pre-trained Convolution Neural Network (CNN) for face detection. This network is based on the ResNet architecture with a Single Shot Detector (SSD) for the detection phase.

This approach uses OpenCV's Deep Neural Network (dnn) module with a pre-trained CNN based on the ResNet architecture and a Single Shot Detector (SSD) for face detection in real-time images.

The CNN processes the input image by applying multiple layers of convolution, pooling, and non-linear activation functions to extract features that represent important aspects of the image. The SSD then takes these feature maps as input and generates bounding boxes around detected faces based on their probabilities and coordinates. Optimized architectures like SSDs enable real-time face detection in video streams due to their efficient single-pass approach.

The pre-trained ResNet model has already been trained on a large dataset of images containing faces, enabling it to recognize and classify different facial features effectively. One limitation is that DNN models can be larger and require more computational resources compared to Haar Cascades. This may not be ideal when resources are scarce.

`detect_faces_dnn(img)`: This function takes an image as input and performs face detection using a DNN model. It:

1. Uses `cv2.dnn.readNet` to load a pre-trained face detection model from a .pb weight file and a .pbtxt configuration file [3](`opencv_face_detector_uint8.pb` and `opencv_face_detector.pbtxt`). [4]
2. Reads the image dimensions (height, width).
3. Converts the image to a blob format suitable for the DNN model, using `cv2.dnn.blobFromImage`.
4. Feeds the blob through the DNN for face detection, using `net.setInput`. The network performs forward propagation using `net.forward` to generate detections.
5. Iterates through detections and draws bounding boxes around faces with confidence scores above a threshold (0.5).
6. Captures video from the webcam using `cv2.VideoCapture(0)`. In a loop, it reads each frame, detects faces, and displays the resulting frame with bounding boxes in real-time.

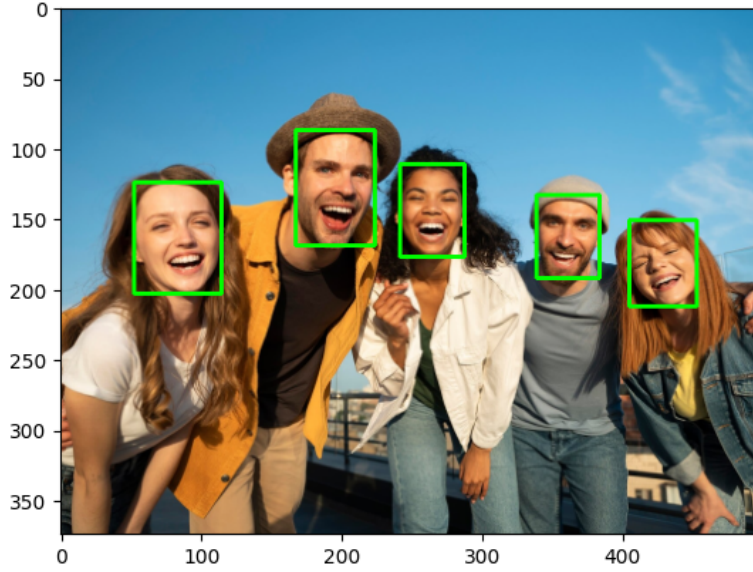


Figure 3: Face detection with DNN classifier

By observing the drawn rectangles in the test image, we verify that this approach has no errors of classification. Compared to Haar Cascades, DNN models often achieve superior accuracy in face detection tasks. They can handle variations in pose, lighting, and occlusions more effectively.

iii. dlib toolbox

Dlib is a toolkit containing machine learning algorithms and tools [5]. It has a set of methods to perform face and facial landmark detection, including up to 68 facial feature points estimation. The face detector uses the Histogram of Oriented Gradients (HOG) feature combined with a linear classifier.

This approach uses OpenCV and dlib libraries to implement real-time object detection in images collected by a camera. OpenCV is employed to handle video capture from the camera and display the resulting frames, while Dlib's machine learning algorithms and tools are used for face detection and facial landmark estimation. The face detector uses Histogram of Oriented Gradients (HOG) features combined with a linear classifier to detect faces in real-time. Once a face is detected, its facial landmarks are estimated using another Dlib function.

1. The detector is loaded through `dlib.get_frontal_face_detector()`, which retrieves a pre-trained model for frontal face detection.
2. The predictor is loaded through `dlib.get_frontal_face_detector()`, using the file `shape_predictor_68_face_landmarks.dat`. This function loads

a pre-trained model for predicting 68 facial landmark points on a detected face.

3. `detect_faces_dlib()` takes an image frame as input and performs face detection and landmark localization. It uses `detector(frame, 1)` to detect faces in the frame. The 1 argument specifies the number of image upsampling levels used to enhance detection for smaller faces. Then, the function iterates through each detected face rectangle with `predictor(frame, rect)` to predict facial landmarks (68 points) on the detected face.



Figure 4: Face detection with Dlib classifier

By observing the drawn rectangles, we verify that this approach has no errors in the test image. We can also observe that this model is able to retrieve face landmarks correctly.

iv. Mediapipe

The MediaPipe Face Detector [6] task detects faces in an image or video. This task can locate faces and facial features within a frame. This task uses a machine learning (ML) model that works with single images or a continuous stream of images. The task outputs face locations, along with the following facial key points: left eye, right eye, nose tip, mouth, left eye tragion, and right eye tragion.

This approach uses OpenCV and MediaPipe libraries for real-time face detection in images collected by a camera. The MediaPipe model uses deep learning

algorithms to analyze video frames or images and identify faces present within them. It provides the position of faces along with their facial key points such as left eye, right eye, nose tip, mouth, left eye tragion, and right eye tragion. These key points can be used for various applications like facial recognition, emotion detection, etc.

The camera is initialized to capture images. The function `detect_faces_mediapipe()` processes each frame using the initialized model and draw the detected faces on the image in real-time:

1. Creates a context manager using with `mp_face_detection.FaceDetection()` to ensure proper resource management for the face detector model.
2. Inside the context manager, the frame is converted to RGB format (expected by MediaPipe) using `cv2.cvtColor`. Face detection is performed using `face_detector.process(rgb_frame)`. This returns a detection result object.
3. For each detected face, `face.location_data.relative_bounding_box` outputs the relative coordinates to absolute pixel values based on the frame size using `np.multiply`. Key facial landmarks (also relative) are retrieved from `face.location_data.relative_keypoints`.

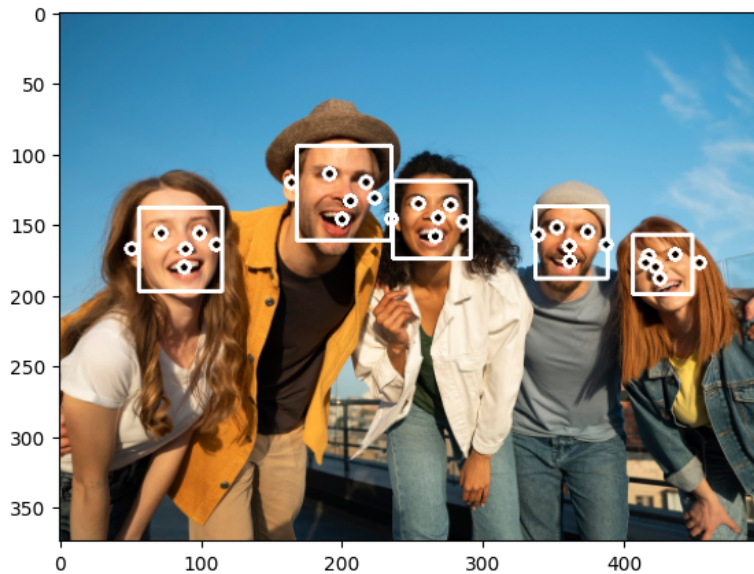


Figure 5: Face detection using Mediapipe

By observing the drawn rectangles, we verify that this approach has no errors in the test image. We can also verify a correct detection of eyes, nose, mouth and ears positions of each face.

b. Face Recognition

i. Face database

Perform the acquisition of a face database from the students in the class (7 face images for each student). This set will be used in this project as the training and test sets. It may also be used as a face database available on the Internet.

The chosen dataset is called "Face Recognition Dataset" [5], and consists in images of 31 celebrities. There is a folder with the original images (**faces/Original Images**), which have different resolutions, angles and framing, but there's also a folder with already normalized images (160x160), which we only need to re-size to the desired size (46x56), and use in the next steps. This could be done to avoid error propagations, because from the exercise of face normalizing can result errors and, if those images with errors are used in the next steps, they can deteriorate the performance of Eigenfaces and Fisherfaces. However, the next exercise generates the normalized images, which will be used in the next steps.

ii. Face normalization

Proceed with the normalization of the face images from the training set (rotation, scaling, selection) so that the faces comply with the format defined by MPEG-7 recommendation. That is, each face is represented by a monochrome image (256 levels) with 56 rows and 46 columns, with both eyes, right and left, perfectly aligned horizontally, and located in line 24, columns 16 and 31, respectively. For eye detection, Haar Cascade classifier can also be used with proper XML classifiers files.

The given Python code performs the following tasks to load and normalize face images from a database using OpenCV and Numpy libraries, adhering to the MPEG-7 recommendation for format and eye alignment:

1. Load the Haar Cascade classifier for detecting eyes (eye_cascade) and faces (face_cascade).
2. Read the input image (image) using OpenCV's imread() function.
3. Convert the input image to grayscale using OpenCV's cvtColor() function.
4. Detect faces in the grayscale image using OpenCV's detectMultiScale() function with face_cascade.
5. For each detected face, detect eyes using OpenCV's detectMultiScale() function with eye_cascade.
6. If two eyes are detected for a face, align both eyes horizontally by swapping the pixels between their positions in the face image.
7. Append the aligned face to a list called 'face_database'.

8. Repeat steps 4-7 for all faces in the input image.
9. Normalize the face images in the 'face_database' list using OpenCV's `resize()` function and Numpy's `mean()` function.
10. Return the normalized face database as output.

The MPEG-7 recommendation for format includes loading grayscale images, detecting faces using a Haar Cascade classifier, and normalizing the faces to a fixed size. The eye alignment step is an additional requirement specific to this code implementation.

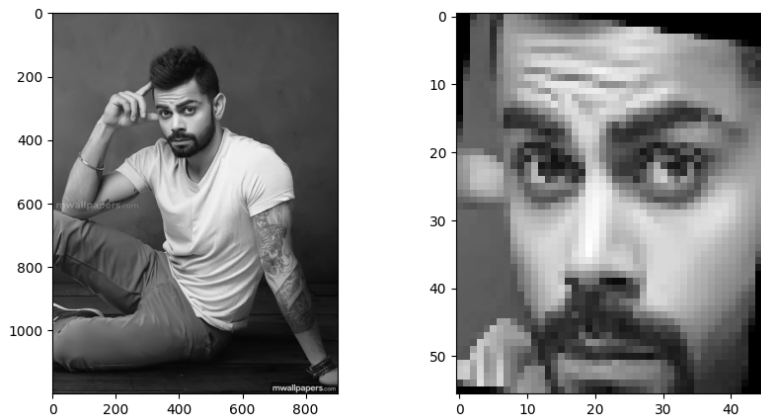


Figure 6: Normalized face image.

iii. Using EigenFaces

1. Compute the mean face vector (column vector with size of 2576×1) and the transformation matrix W_{pca} (with size of $2576 \times m$), containing the first m eigenfaces (where m can be a number between 10 and 20). Use the procedures presented in the slides of the course. View the $m+1$ obtained vectors by interpreting them as face images (eigenface) and prove that they form an orthogonal basis.
2. Perform the projection of some of the training set faces on the face subspace and observe their reconstructions. Compute (and view) the error face (difference between the original face and its reconstruction). Show that the error face is orthogonal to the face subspace.

Eigenfaces is an unsupervised method that uses Principal Component Analysis (PCA) to find the principal components in the data. It analyzes the covariance matrix of the training set and finds eigenvectors corresponding to the largest eigenvalues. These eigenvectors represent the directions of maximum variance in the data, and the coefficients of these eigenvectors for each image form a new feature vector. The number of eigenvectors kept determines the dimensionality

of the lower-dimensional space.

The method `fit()`, from `EigenfacesModel` object, trains the model on a set of training images (`Xtrain`) through the following steps:

1. **Mean face** - Resizes and flattens each training image into a vector, to calculate the mean face by averaging all training image vectors.
2. **Centered Faces (A)** - Subtracts the mean face from each training image vector to obtain centered faces.
3. **Covariance Matrix (R)** - Computes the covariance matrix (R) by multiplying the transpose of A with A. This captures the relationships between the centered faces.
4. **Eigenvectors and Eigenvalues** - Uses `np.linalg.eig` to find the eigenvectors and eigenvalues of the covariance matrix R. Eigenvectors represent the principal directions of variance in the data, while eigenvalues indicate the significance of each direction.
5. **Selecting Top Eigenfaces (V)** - Chooses the top m eigenvectors corresponding to the highest eigenvalues. These capture the most important variations in the faces and reduce dimensionality.
6. **Weights (W)** - Calculates the weights (W) by multiplying the centered faces (A) with the selected eigenvectors (V). These weights represent how much each eigenvector contributes to reconstructing a particular face.
7. **Normalized Weights (W_{norm})** - Normalizes the weights (W) by dividing each row by its norm. This ensures weights have a unit length and contribute equally in the reconstruction process.
8. **Projections** - Projects the centered faces onto the face space spanned by the top m eigenvectors (V). This results in projections representing the faces in the reduced dimension.

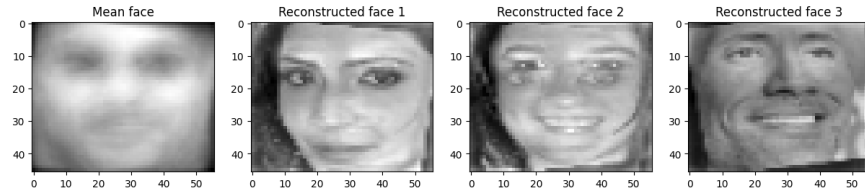


Figure 7: Mean face and reconstructed faces for Fisherfaces algorithm

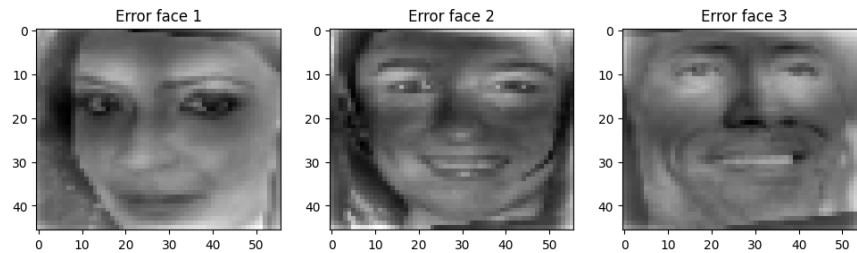


Figure 8: Error faces for Fisherfaces algorithm

iv. Using FisherFaces

Repeat the previous points, but now using the algorithm called fisherfaces. Use the procedures presented in the slides of the course.

Fisherfaces is a supervised method that uses Linear Discriminant Analysis (LDA) to find the features that maximize the between-class variance and minimize the within-class variance. It analyzes the covariance matrices of each class separately and then combines them into a Fisher matrix. The eigenvectors corresponding to the largest eigenvalues of this Fisher matrix are used as new feature vectors. The Fisherfaces Model builds upon the Eigenfaces Model by:

- Leveraging class labels during training to separate between different classes.
- Using the Within-class (S_w) and Between-class (S_b) scatter matrices to focus on variations that distinguish classes.
- Employing Fisherfaces, which are eigenvectors derived from the projected scatter matrices, which allow better class discrimination.

The method `fit()`, from `FisherfacesModel` object, trains the model on a set of training images (`Xtrain`):

1. Resizes and flattens each training image into a vector.
2. Creates an instance of `EigenfacesModel` (`pca`) with `m` eigenfaces. Trains the `Eigenfaces` model on the training data (`Xtrain`, `ytrain`). This step computes the mean face, centered faces (`A`), and the top `m` eigenvectors (`Wpca`).
3. Calculates the mean face for each class (represented by unique labels in `ytrain`).
4. Computes the difference between each class mean and the overall mean face.
5. Initializes two scatter matrices: Within-class scatter matrix (S_w), which captures the variations within each class, and Between-class scatter matrix (S_b) that captures the variations between different classes.

6. For each class, calculates the difference between class faces and their class mean.
7. Updates S_w by adding the dot product of the centered faces for each class. Updates S_b by adding the dot product of the class difference vectors.
8. Reuses the eigenvectors (V_{pca}) computed by the Eigenfaces model for dimensionality reduction. Projects both S_w and S_b matrices onto the subspace spanned by the top m eigenvectors (W_{pca}). This reduces the dimensionality while preserving relevant information.
9. Computes the inverse of the projected S_w matrix (assuming it's invertible). Calculates the product of the inverse S_w and the projected S_b matrix.
10. Finds the eigenvectors and eigenvalues of this resulting matrix. Sorts the eigenvalues in descending order and selects the top $\text{num_classes} - 1$ eigenvectors (corresponding to the largest eigenvalues). These represent the Fisherfaces.
11. Combines the selected eigenvectors with the PCA eigenvectors (W_{pca}) to form the Fisherfaces basis (W_{fisher}). Projects the centered faces (A) onto the Fisherfaces space using the transposed Fisherfaces basis (W_{fisher}^T). This gives the representations of faces in the reduced Fisherfaces space (projections).

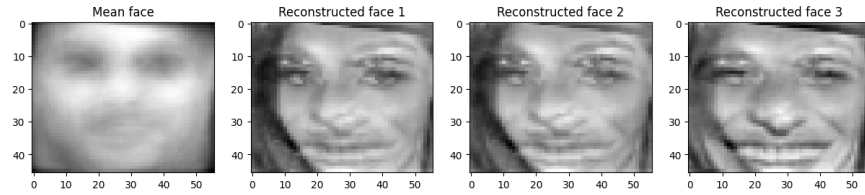


Figure 9: Mean face and reconstructed faces for Fisherfaces algorithm

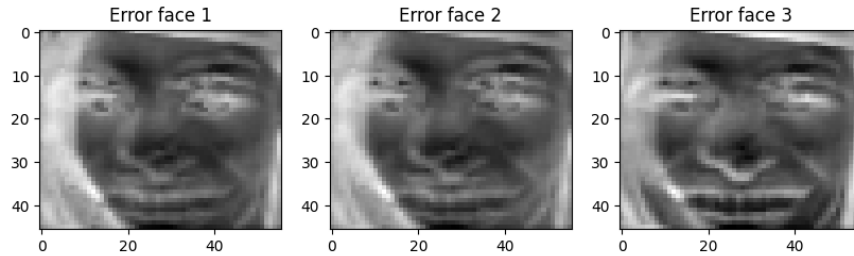


Figure 10: Error faces for Fisherfaces algorithm

v. Classification

Develop a Nearest-Neighbor (NN) classifier based on the feature vectors resulting from the m projection coefficients. Consider an arbitrary number of classes, c (involving all students in the class or just a part of them). Test the implemented classifiers using the test set (different from the training set).

To recognize faces with each algorithm, a Nearest Neighbour classifier can be developed, which consists in measuring the distance from the test projection, i.e. the face to recognize, to the feature vectors resulted from the m projection coefficients, and then pick up the face in the face subspace that's more "alike" according to the algorithm.

The train and test sets are produced using `train_test_split()` function from Sklearn, with a test portion of 30%.

The `predict()` function for the `EigenfacesModel` class, predicts the identity of a face in a test image:

1. Resizes the test image to a consistent size (46x56 pixels) to match the training data.
2. Subtracts the mean face from the flattened test image to obtain the centered test image. This removes the average face information and focuses on variations specific to the test image.
3. Projects onto face space, by performing the dot product between the transpose of W and the centered test image.
4. Perform Nearest Neighbour search, by calculating the Euclidean distance between the test projection and the current training face projection using `np.linalg.norm`. Euclidean distance measures the "straight-line" distance between two points in this high-dimensional space. Other metrics can be suitable.

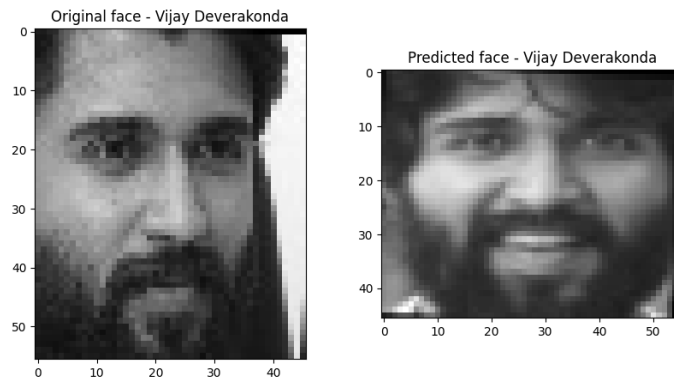


Figure 11: Face recognition with Eigenfaces using image from Xtest

The `predict()` function for the `FisherfacesModel` class, predicts the identity of a face in a test image:

1. Resizes the test image to a consistent size (46x56 pixels) to match the training data.
2. Subtracts the mean face from the flattened test image to obtain the centered test image. This removes the average face information and focuses on variations specific to the test image.
3. Projects onto face space, by performing the dot product between the transpose of `Wfisher` and the centered test image.
4. Perform Nearest Neighbour search, by calculating the Euclidean distance between the test projection and the current training face projection using `np.linalg.norm`. Euclidean distance measures the "straight-line" distance between two points in this high-dimensional space. Other metrics can be suitable.

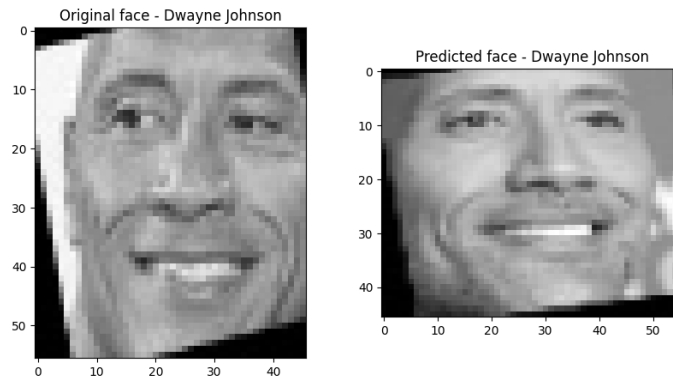


Figure 12: Face recognition with Fisherfaces using image from Xtest

c. Combine real and virtual objects.

i. Virtual object normalization

Based on the alignment of section b.ii, normalize the virtual object that is intended to be superimposed on the identified face, for example, a hat, glasses, or other object of your choice. Use different objects for different faces/persons.

ii. Adding with mask

Add to the image, using a mask.

In this approach, normalization and superimposition of a hat and glasses was achieved in images and real-time video through webcam.

superimpose_hat() and superimpose_glasses():

- Loads a hat or glasses image from the './masks' directory.
- Resizes the hat/glasses image based on the face dimensions ('w' and 'h') and a user-defined 'scale_factor'.
- Calculates the placement coordinates: for the hat on top of the face, and for the glasses, a little below (adjusted by percentages of 'h').
- Iterates through each pixel of the hat/glasses image.
- If the pixel value is greater than 0 (skipping transparent areas), it copies the color value to the corresponding pixel in the face image. This is needed because the of the transparent background being black.
- Returns the modified image with the superimposed object.

In the `c_i.ii_mask.py` file we can see the program executing in real time with video from webcam.

This code provides two functions for superimposing accessories (hat and glasses) onto detected faces. It can be used on both static images and live video streams.



Figure 13: Superimposition of normalized hats

For simplification, it was used the Haar Cascade classifier in the `haarcascade_frontalface_default.xml` file, so only 4 faces are identified, as seen in 2.a.ii. We can observe the superimpositions of the virtual objects. The hats look well positioned, the glasses could be more adjusted, but since there are several faces and positions in the image.

3 Conclusion

In this project I had the opportunity to explore, experiment and develop image processing tasks that can be later used in further projects.

Examples of future work:

- System to recognize people by face in real time, using Eigenfaces or Fisherfaces with a database.
- More virtual object superimpositions to several purposes.

References

- [1] Stack Overflow - I can't install python dlib through anaconda. <https://stackoverflow.com/questions/71658399/i-cant-install-python-dlib-through-anaconda/>.
- [2] Viola, P., Jones, M. J. (2001). Rapid object detection using a boosted cascade of simple features. [PDF] <https://ieeexplore.ieee.org/document/990517>
- [3] Elina Maliarsky, Video KYC — Part II <https://medium.com/@elinamaliarsky/video-kyc-part-ii-792129847df4>
- [4] Face detection models DNN https://github.com/opencv/opencv/tree/master/samples/dnn/face_detector
- [5] Vasuki Patel (in Kaggle). Face recognition dataset. <https://www.kaggle.com/datasets/vasukipatel/face-recognition-dataset>, 2023.